

Homework Assignment 2

(Programming Category)

Student Name: _____

Student Session: cs6220

Representational Learning with Word2Vec

The best hands-on way to learn word embeddings in one day is to build and train a Word2Vec model from scratch using Python, then visualize the results. Word2Vec uses a shallow neural network to map words into a dense vector space based on the context in which they appear. There are two main baseline architectures for Word2Vec: **Skip-gram** which predicts context words from a target word; and **CBOW** (Continuous Bag of Words) which predicts a target word from context.

- **Training Speed:** [CBOW](#) is several times faster to train because it averages context words into a single vector prediction. Skip-gram treats every context-target pair individually, making it slower.
- **Data Size:** [Skip-gram](#) performs better with smaller amounts of training data. CBOW requires large datasets to perform at its best.]
- **Accuracy on Words:** CBOW works better for frequent and common words. Skip-gram represents rare words, unusual phrases, and subtle relationships much more accurately.

In this HW2 programming option, you are asked to gain hand-on experience on representational learning with a Word2Vec model.

You can download word2vec implementation from your favorite ML library, and feel free to choose Skipgram or CBOW. Here are some examples.

GENSIM <https://radimrehurek.com/gensim/models/word2vec.html>

Keras implementation of word2vec:-<https://github.com/abaheti95/Deep-Learning/tree/master/word2vec/keras>

Pytorch implementation word2vec <https://github.com/Andras7/word2vec-pytorch>

Google Code: <https://code.google.com/archive/p/word2vec/>

In this programming problem, you learn word2vec by taking take a superficial look at the code, such as the Python implementation in Gensim (radimrehurek.com/gensim/index.html) API at <http://radimrehurek.com/gensim/models/word2vec.html>.

Here are some steps to **Build a Model with Gensim**

- Set up a Python environment and load a text corpus (such as movie reviews, product reviews, or a small Wikipedia dump).
- Use the Gensim Library to preprocess your text and train a custom Word2Vec

model. You may find this [tutorial](#) and the [example](#) helpful.

Here are steps for **Query & Test & Visualize Your Embeddings**

- Write code to find the most similar words using `model.wv.most_similar()`.
- Experiment with vector math analogies (e.g., *King - Man + Woman = Queen*).
- **Visualize the Vector Space:** Use **t-SNE** via scikit-learn to reduce your high-dimensional vectors down to 2 dimensions or use **PCA** (Principal Component Analysis). Plot the words on a scatter chart to see semantically similar words cluster together naturally.

You may use the [Stanford Sentiment Treebank \(SST\)](#) dataset to train your own word2vector models, or to take a pretrained model on SST and fine tune this pretrained model through transfer learning on your self-created dataset. Then apply your model to a simple sentiment analysis task using test data. You may also create your own dataset for training a word2Vec encoder-decoder by yourself.

Deliverable: In your HW2 report,

1. you are asked to compare the performance of **two different Word2Vec models**. For each, describe the word2vec model in terms of its neural architecture, incl. all layers from input, encoder layers, code layer, decoder layers to output) using a figure. Ideally, both or at least one of the Word2Vec models is trained or finetuned by yourself. **[Hints:]** If you could not build one yourself, you could compare your test results on two different pre-trained models with the test dataset you choose. But you are required to show your understanding on the hyperparameters and architectures of the two models you downloaded to run the tests.
2. you are asked to describe the total training dataset used, and the total test dataset used and provide them in a subdirectory when you zip your HW submission.
 - a) For pretrained model(s) you downloaded, provide reference and URL. Also include the reported test accuracy and provide the training dataset and test dataset with URL links
 - b) For your own fine-tuned model from a pretrained Word2Vec model, provide 5 representative training examples used for finetuning and 4 representative test examples (2 positive examples and 2 negative examples).
 - c) Provide a performance measurement table with your analysis. Include representative test examples, in those examples, you should have at least 2 positive examples on which your Word2Vec model did correctly, and 2 negative examples on which your Word2Vec model did incorrectly.

Evaluation Requirement: If your test data is too simple, all test results are positive for your own trained models or for the pre-trained models you downloaded, then you need to add more creative examples, until you have sufficient negative examples under your tests to report under (i) your own trained models and (ii) under the pre-trained two auto-encoder(s).

In addition to the above information/data in your report for HW2 programming option, you are asked to provide the following itemized deliverables:

3. Report your trained programming HW2 model by showing the top K ranked

results for 5 test examples. Compare your model with at least one pre-trained open source models for your chosen auto-encoder application (e.g., sentiment analysis) using your test data created or downloaded from public benchmarks. Elaborate/discussion your comparison results.

4. Your deliverable should include the performance measurement results for model training, for model testing (average per query performance), and comparison of the two Word2Vec models you use for this HW2.
5. Analyze the two Word2Vec models in terms of hyper-parameters, e.g., context window size or loss function or other hyperparameters like learning rate, etc., and compare your two different word2vec models using top-1, top-5 and top-10 ranked words for five different word queries.
6. Your report may also include the experience with the installation, running and measurement. You can provide your deliverable in ppt or word or pdf.